ADVANCED BEAUTRY CARE

AI POWERED SKINCARE ANALYSIS SYSTEM

TECHNICAL DOCUMENTATION

Developed BY

MARY ALICE

React.js • Flask • Python • YOLOv8 OpenCV • MediaPipe • Firebase

---

Intelligent Analysis • Personalized Care • Healthier Skin

Version 1.0

2026

Document Information

Document Details

Field	Information
Document Title	Advanced Beauty Care – Technical Documentation
Project Name	Advanced Beauty Care
Project Type	AI-Powered Skincare Analysis System
Document Version	1.0
Release Date	July 2026
Prepared By	Mary Alice
Project Status	Completed
Document Status	Final
Language	English
Category	Artificial Intelligence & Computer Vision

Intended Audience

This documentation is intended for:

- **Developers**
- **Recruiters**
- **Project Evaluators**
- **Students**
- **Software Enthusiasts**

Document Scope

This document provides technical information about the Advanced Beauty Care system, including system architecture, implementation, AI models, application modules, testing, results, and future enhancements. It serves as a technical reference for understanding the project's design and functionality.

Abbreviations

Abbreviation	Full Form	Abbreviation	Full Form
AI	Artificial Intelligence	**DB**	Database
CV	Computer Vision	**SQL**	Structured Query Language
ML	Machine Learning	**OCR**	Optical Character Recognition
UI	User Interface	**JSON**	JavaScript Object Notation
UX	User Experience	**HTTP**	Hypertext Transfer Protocol
API	Application Programming Interface	**CSS**	Cascading Style Sheets
REST	Representational State Transfer	**HTML**	HyperText Markup Language
YOLO	You Only Look Once	**JS**	JavaScript

1. Introduction

The skincare industry is one of the fastest-growing sectors in the global consumer market, valued at over USD 180 billion and projected to surpass USD 270 billion by 2030. Despite this explosive growth, a significant gap remains between the availability of skincare products and consumers' ability to make informed, personalised choices about what is right for their individual skin. Most people rely on generic advice from friends, social media influencers, or product packaging — none of which account for their specific skin type, concerns, tone, or environmental factors.

Professional dermatological consultations, while accurate, are expensive, time-consuming, and inaccessible to the majority of the population, particularly in developing countries such as India where the dermatologistto-patient ratio remains critically low. This creates a vacuum that technology is uniquely positioned to fill.

Advanced Beauty Care is a full-stack, AI-powered web application that brings professional-grade skin analysis to every smartphone or computer with a camera. By simply uploading a face photograph, users receive a comprehensive skin health report that includes skin type classification, acne detection (type and severity), and a quantified breakdown of skin concerns such as oiliness, redness, dryness, pigmentation, and uneven texture. The system then recommends curated skincare products and explains every ingredient in plain language — empowering users to understand not just what to use, but why.

The system is built on a modern technology stack: a React frontend deployed globally via Cloudflare Pages, a Flask-based backend running deep learning inference on Google Colab, and Firebase for authentication and real-time database storage. The AI models — three separate YOLOv8 classifiers for skin type, acne type, and acne severity — are complemented by a custom pixel-level analysis engine using MediaPipe FaceMesh and LAB/HSV colour space mathematics to compute quantitative skin metrics.

A particularly important innovation in this project is the implementation of skin-tone-fair redness detection. Most existing skin analysis tools use raw RGB colour channels to estimate redness, which systematically over-flags darker skin tones as "sensitive" due to their lower baseline brightness — a well-documented bias in dermatological AI. Advanced Beauty Care eliminates this bias by using the LAB colour space's a* channel, which measures red-green opposition independently of skin brightness, producing accurate redness readings across the full spectrum of human skin tones from Fair to Deep.

1.1 Problem Overview

The fundamental problem this project addresses is the lack of accessible, accurate, and transparent skin analysis for the general public. Several specific challenges compound this problem. First, there is a widespread lack of personalized skin analysis tools. Generic quizzes on beauty websites ask a handful of subjective questions and return broad categorizations that do not reflect the nuanced reality of individual skin. A person may be oily in the T-zone but dry on the cheeks, may have pigmentation alongside sensitivity, or may have

acne that is hormonal rather than hygiene-related — and these distinctions matter enormously for product selection.

Second, existing product recommendation engines are driven primarily by marketing logic rather than dermatological logic. They are designed to maximize sales of high-margin products rather than to identify the most effective solution for a given skin condition. There is rarely any explanation of why a particular ingredient is beneficial for a particular concern, leaving consumers unable to evaluate whether a recommendation makes sense.

Third, and perhaps most critically, existing AI-powered skincare tools exhibit a significant bias against darker skin tones. This is not a minor inconvenience — it means that millions of users with medium to deep skin tones receive inaccurate diagnoses that may lead them to purchase unnecessary or even harmful products. The root cause of this bias, and the technical solution implemented in this project, is discussed in detail in the Methodology section.

Fourth, the skincare ingredient landscape is bewildering for most consumers. Products list dozens of chemical names — niacinamide, hyaluronic acid, retinol, salicylic acid, ceramides — with no explanation of what they do, how they work, or whether they are appropriate for a given skin type. Advanced Beauty Care addresses this through a searchable ingredient encyclopedia that explains every ingredient in the system in plain language, cross-referenced with the products that contain it.

1.2 Objectives

The primary objectives of the Advanced Beauty Care system are listed below. The first objective is to develop an AI-powered web application that analyses facial skin from a photograph, making personalized skin assessment accessible to any user with a smartphone or computer camera. The second is to classify skin type across five categories — Normal, Dry, Oily, Combination, and Sensitive variants — using a trained YOLOv8 deep learning model.

The third objective is to detect acne type (Papules, Pustules, Nodules, Blackheads, Whiteheads) and severity (Mild, Moderate, Severe) using a second and third YOLOv8 model respectively, trained on a curated acne dataset. The fourth objective is to compute five quantitative skin metrics — oiliness, redness, dryness, pigmentation, and texture — using MediaPipe FaceMesh landmark zones and LAB/HSV color space analysis, providing a more objective and reproducible skin assessment than purely subjective questionnaires.

The fifth objective is to provide a 0–100 skin health score with identified skin issues presented in clear language. The sixth is to deliver personalized product recommendations with per-ingredient explanations tailored to the user's specific skin profile. The seventh, and most technically significant, is to implement skintone-fair redness detection using the LAB a* channel, specifically eliminating the bias against darker skin tones documented in earlier model iterations. The eighth objective is to allow users to manually input skin concerns as an alternative to the face scan — ensuring accessibility for users who are uncomfortable with

image analysis. The ninth is to maintain user scan history and enable trend tracking via Firebase Firestore, and the tenth is to deploy a stable, production-ready system accessible via any modern web browser.

2. Problem Analysis

The development of Advanced Beauty Care follows a structured approach to understand and address the challenges faced in personalised skincare. The system is designed to provide an integrated solution where users can analyse their skin, receive personalised recommendations, track progress, and manage a skincare routine — all from a single platform.

The analysis focuses on how image data flows through the system, from upload and validation through AI inference and pixel analysis, to generating meaningful personalised outputs. The system applies rule-based logic and machine learning models to analyse skin condition and provide insights such as skin type classification, issue detection, and product recommendations.

2.1 METHODOLOGY

The development of Advanced Beauty Care follows an Iterative Software Development Model, where the system is built in stages with continuous testing and improvements. This approach allows each module to be developed and verified independently before integrating into the complete system.

The system is designed using a three-tier architecture:

- Frontend (React 18 + Vite) – Provides an interactive and user-friendly interface for photo upload, validation feedback, analysis results, product browsing, and progress tracking.
- Backend (Flask + Python on Google Colab) – Handles AI model inference, image preprocessing, pixel analysis, makeup detection, and API communication.
- Database (Cloud Firestore) – Stores user scan history, authentication data, and dynamic configuration (ngrok URL).

The development process is carried out in the following stages:

- Requirement Analysis – Identifying user needs: skin analysis, product recommendations, routine planning, progress tracking.
- Dataset Collection & Model Training – Training three YOLOv8 models on Kaggle datasets with balanced class distributions.
- System Design – Designing modules, database structure, API endpoints, and UI components.
- Implementation – Developing the full-stack application with all modules.
- Testing – Unit testing and integration testing of all modules.
- Deployment – Cloudflare Pages (frontend) and Google Colab + ngrok (backend).

2.2 Process Flow and Data Flow Diagram

The process flow describes how user data is collected, processed, and converted into meaningful skincare insights. The movement of data begins when the user uploads a photograph and ends with a personalised skincare report displayed on the results page.

The Data Flow Diagram of the system is represented at three levels:

- Level 0 DFD (Context Diagram) – Shows the overall interaction between the user and the system.

- Level 1 DFD – Provides a detailed view of internal processes such as image validation, AI inference, pixel analysis, and recommendation generation.
- Level 2 DFD – Provides a detailed breakdown of each internal process, showing how data is handled and transferred within individual modules.

Level 0 DFD (Context Diagram):

The Level 0 DFD represents the overall functioning of Advanced Beauty Care as a single process. The user provides inputs such as a facial photograph or manual questionnaire responses. The system processes these inputs through AI models and pixel analysis, stores scan data in Firebase, and returns outputs including skin analysis reports, product recommendations, AM/PM routines, and progress comparisons.

Entity / Component	Role	Data Flow
User	External entity	Uploads photo → receives analysis report, products, routine
React Frontend	User interface	Sends image to Flask → receives JSON result → renders UI
Flask Backend	Processing engine	Validates image → runs YOLO → pixel analysis → returns result
Firebase / Firestore	Data store	Stores scan history, user auth, ngrok config URL
YOLOv8 Models (×3)	AI inference	Classifies skin type, acne type, acne severity
MediaPipe / OpenCV	CV pipeline	Face detection, landmark zones, pixel metrics

Level 1 DFD:

At this level, the system is divided into major processes:

- Image Upload & Validation – Face detection, anime rejection, makeup detection, quality check.
- AI Inference – Three YOLOv8 models (skin type, acne type, acne severity) run in sequence.
- Pixel Analysis – MediaPipe landmark zones used for oiliness, redness, pigmentation, dryness, texture.
- Recommendation Engine – Matched products, AM/PM routine, home remedies generated from results.
- History & Progress – Scan saved to Firestore; dashboard compares scores across sessions.

Level 2 DFD:

At this level, each major process is broken down further:

- Validation Process – Haar Cascade detects face → variance check rejects anime → MediaPipe FaceMesh checks lipstick, eye makeup, foundation, blush → quality score computed.
- AI Inference Process – Image compressed to 800px → uploaded to Flask → face cropped with 20% padding → CLAHE/denoise/sharpen normalisation → YOLO models run on 512×512 image.
- Pixel Analysis Process – FaceMesh applied to normalised image → skin mask built excluding lips/eyes/brows/bindi → HSV/LAB channels sampled → 5 metrics computed.

- Recommendation Process – Skin type + issues matched against product database → AM/PM routine built → home remedies and dietary advice selected → response JSON assembled.

3. Design Phase

The design phase focuses on organising Advanced Beauty Care in a structured and efficient manner. The system follows a modular approach, where it is divided into distinct functional modules: Authentication, Home/Landing, Image Analysis, Processing, Results, Products, Product Detail, Ingredient Scanner, Manual Input, Recheck/Progress, and Dashboard.

The application is built using a three-tier architecture: React frontend, Flask backend, and Firebase/Firestore for data persistence. The design ensures a clean, user-friendly interface with clear visual hierarchies and a consistent green-toned design system throughout.

3.1 MODULARIZATION

The system is designed using a modular approach, where the entire application is divided into smaller functional units. Each module performs a specific function and interacts with the backend through welldefined API interfaces.

The major modules of the system are:

- Authentication Module – Handles user registration and login via email/password or Google OAuth through Firebase Authentication.
- Home / Landing Module – Provides the entry point with navigation to analysis, manual input, and demo mode.
- Analysis Module – Manages photo upload, drag-and-drop, QR code phone scan, client-side validation, and navigation to the processing pipeline.
- Image Validation Module – Runs face detection (Haar Cascade), anime/cartoon rejection (pixel variance), and makeup detection (MediaPipe FaceMesh) before analysis.
- Processing Module – Displays animated progress with step-by-step status updates while the image is compressed, uploaded, and analysed by the Flask backend.
- Results Module – Renders the full skin analysis report: score ring, skin type, tone, issues, facial region breakdown, metric bars, solution tabs (Steps/Foods/Remedies/Routine), and matched products.
- Products Module – Displays the curated 124-product database with filters by skin type and brand, cart and wishlist functionality.
- Product Detail Module – Shows full product information, key ingredient cards with mechanism explanations, usage instructions, and customer reviews.
- Ingredient Scanner Module – Searchable database of skincare ingredients with benefit, mechanism, and products containing each ingredient.
- Manual Input Module – 7-step questionnaire wizard that generates an offline personalised skincare plan without requiring a photo.
- Recheck / Progress Module – Displays scan history with score tracking and routine checkup analysis.
- Dashboard Module – Shows scan history timeline with skin score progress graph, cart, wishlist, and settings.

3.2 Entity-Relationship Diagram and Class Diagram

The data model of the system comprises six key entities. The User entity represents a registered account and stores the Firebase Authentication UID, email address, display name, and account creation timestamp. A User has a one-to-many relationship with ScanRecords, as each user can perform multiple scans over time.

The ScanRecord entity captures the complete result of a single skin analysis session. Its key fields include: scanId (auto-generated Firestore document ID), uid (foreign key to the User), timestamp (ISO 8601 datetime), skinType (the classified skin type string), score (integer from 20 to 100), issues (array of detected concern strings), acneType, acneSeverity, tone (the detected skin tone category), and the five quantitative metrics (avgOil, avgRed, avgDry, avgPig, avgTex) as floating-point values between 0.0 and 1.0. An optional imageUrl field stores a Firebase Storage reference to the analysed photograph for display in the scan history.

The Product entity represents a skincare product in the recommendation catalogue. Each product has a unique ID, name, brand, category (cleanser, moisturiser, serum, etc.), price, a list of compatible skin types, a list of target issues it addresses, an ordered list of ingredient names, usage instructions, and a description. Products maintain a many-to-many relationship with Ingredients via the ingredient name list, and appear in multiple Wishlists and Carts across different users.

The Ingredient entity stores educational data about each active ingredient: its name, primary benefit summary, a scientific explanation of how it works at the cellular level, and an array of skin types it is suitable for (goodFor[]). The Wishlist and Cart entities are join entities linking a User to a set of Products, with Cart additionally storing a quantity field per item.

FIGURE: DATA MODEL ER DIAGRAM FOR SKINCARE APPLICATION
Entities: User, ScanRecord, Product, Ingredient, Wishlist, Cart

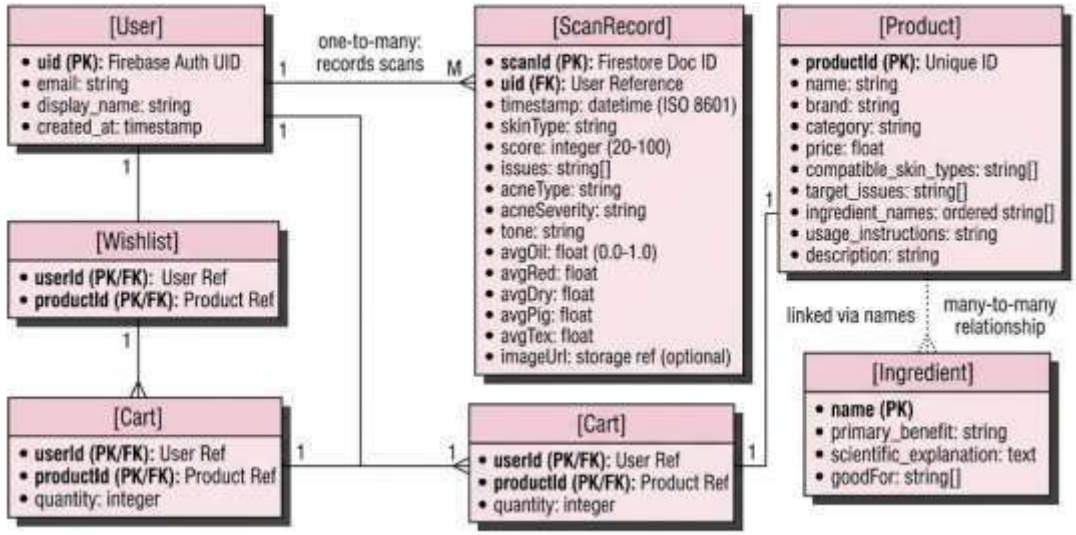

Figure: Database Schema (ER Diagram) for Skincare System

3.3 Database Design

The database for Advanced Beauty Care uses Cloud Firestore (NoSQL document database) for storing user data and scan history. Firestore was chosen for its real-time capabilities, offline support, and tight integration with Firebase Authentication.

1. Users Collection (managed by Firebase Auth)

Field	Type	Description
uid	String (PK)	Firebase Auth unique user identifier
email	String	User email address
displayName	String	User display name
provider	String	google.com or password

2. scanHistory Collection (Firestore)

Field	Type	Description
id	String (PK)	Auto-generated Firestore document ID
uid	String (FK)	Reference to Firebase Auth user
date	String	Formatted date of scan (DD Mon YYYY)
time	String	Time of scan (HH:MM)
skinType	String	Detected skin type (e.g. Sensitive)
issues	Array<String>	List of detected skin issues
score	Integer	Skin health score (0–100)
tone	String	Detected skin tone
conf	Float	Analysis confidence (%)
analysis	Object	Full analysis result object
imageUrl	String	Base64 encoded scan image or null

3. config Collection (Firestore)

Document	Field	Description
colab	apiUrl	Current ngrok tunnel URL for Flask backend
colab	updatedAt	Timestamp of last URL update

3.4 DATA INTEGRITY

Data integrity in Advanced Beauty Care is maintained through the following mechanisms:

- Image Validation – Every uploaded image passes through a multi-stage validation pipeline before analysis: face detection, anime rejection, lighting quality check, aspect ratio check, and makeup detection. Invalid images are rejected with clear error messages.
- Quality Threshold – Images with a quality score below 40/100 (computed from blur level, brightness, and resolution) are rejected before YOLO inference.
- Frontend Validation – The React frontend validates file types (PNG/JPG/WEBP), enforces a 30second timeout with graceful fallback, and compresses images before upload to prevent server errors.
- Firestore Rules – Firebase Security Rules ensure that users can only read and write their own scan history documents (uid-scoped access).
- Fallback Handling – If the Flask backend is unreachable, the system falls back to a client-side pixel analysis using the HTML5 Canvas API. Results are labelled as offline analysis to prevent confusion.
- Overpayment Prevention – N/A for skincare, but equivalent data consistency: scan scores are bounded between 0–100 and confidence values between 0–100 at computation time.

3.5 DATA DICTIONARY

The data dictionary defines the key attributes used across the system, their types, and their purpose:

Attribute	Type	Module	Purpose
skinType	String	Result / History	Classified skin type from YOLOv8 (Oily/Dry/Normal/Combination/Sensitive)
acneType	String	Result	Acne classification from YOLOv8 (Blackheads/Cystic/Papules/Pustules/Whiteheads)
acneSeverity	String	Result	Severity from YOLOv8 (Mild/Moderate/Severe)
score	Integer (0-100)	Result / Dashboard	Overall skin health score computed from issues and pixel metrics
conf	Float (0-100)	Result	Average confidence across 3 YOLO models
avgOil	Float (0-1)	Result / Backend	Oiliness metric from HSV saturation channel
avgRed	Float (0-1)	Result / Backend	Redness metric from LAB a* channel (skin-tone agnostic)
avgDry	Float (0-1)	Result / Backend	Dryness metric from inverted HSV value channel
avgPig	Float (0-1)	Result / Backend	Pigmentation metric from LAB L* channel ratio
avgTex	Float (0-1)	Result / Backend	Texture metric from Laplacian standard deviation
issues	Array<String>	Result / History	List of detected skin concerns
tone	String	Result	Skin tone classification (Fair/Light/Medium/Wheatish/Dusky/Deep)
regStatus	Object	Result	Per-region status (Forehead/Cheeks/Nose/Chin)
source	String	Result	'api' = Flask backend used, 'pixel' = offline fallback

3.6 Software Tools

The development of Advanced Beauty Care utilises the following software tools and technologies:

Component	Technology	Purpose
Frontend	React 18 + Vite	Dynamic, component-based user interface with fast HMR development
Frontend Deployment	Cloudflare Pages	Global CDN hosting for React app with automatic deployments
Backend Framework	Flask (Python)	REST API for AI inference, image processing, and session management
Backend Hosting	Google Colab (GPU)	Free GPU runtime for YOLOv8 model inference
Tunnel	ngrok	Exposes Colab Flask server to public internet with stable URL
AI Models	YOLOv8 (Ultralytics)	Image classification for skin type, acne type, and severity
Computer Vision	OpenCV	Image preprocessing (CLAHE, denoise, sharpen) and pixel analysis
Face Analysis	MediaPipe FaceMesh	468-landmark face mesh for zone-based analysis and makeup detection
Authentication	Firebase Auth	Email/password and Google OAuth user management
Database	Cloud Firestore	NoSQL document database for scan history and config
Styling	CSS Variables + Custom	Green-toned design system with glassmorphism cards
Charts	Custom Canvas / SVG	Score ring visualisation and skin metric bars
QR Code	qrcode-generator (CDN)	Client-side QR code generation for phone scan feature

4. Coding

The coding phase involves implementing the system using Flask (Python) for the backend and React 18 for the frontend. The application is developed using an API-based architecture where each module is handled through specific Flask routes and React components.

4.1 BACKEND — FLASK (flask_predict_final.py)

Image Preprocessing Pipeline

The normalize_image() function applies a three-stage preprocessing pipeline before YOLO inference:

```
def normalize_image(img, size=(512, 512)):
    # Stage 1: Denoise     img = cv2.fastNlMeansDenoisingColored(img,
None, h=8, hColor=8,                templateWindowSize=7,
searchWindowSize=21)     # Stage 2: CLAHE — normalise contrast
without altering colour     lab = cv2.cvtColor(img,
cv2.COLOR_BGR2LAB)     l, a, b = cv2.split(lab)     l =
cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8,8)).apply(l)     img =
cv2.cvtColor(cv2.merge([l,a,b]), cv2.COLOR_LAB2BGR)
```

```
    # Stage 3: Sharpen     kernel = np.array([[0,-0.5,0],[-0.5,3,-0.5],[0,-
0.5,0]])     img = np.clip(cv2.filter2D(img,-1,kernel), 0,
255).astype(np.uint8)     return cv2.resize(img, size,
interpolation=cv2.INTER_LANCZOS4)
```

This function applies denoising to remove sensor noise from phone camera JPEGs, CLAHE (Contrast Limited Adaptive Histogram Equalisation) to normalise lighting variations across different environments, and a sharpening kernel to enhance skin texture details needed for accurate YOLO inference and texture analysis.

Skin Issue Analysis using LAB Color Space

The analyze_skin_issues() function computes 5 skin metrics from pixel data using MediaPipe landmark-based zone sampling:

```
def analyze_skin_issues(img_bgr, lms=None, W=None,
H=None):     img_hsv = cv2.cvtColor(img_bgr,
cv2.COLOR_BGR2HSV)     img_lab = cv2.cvtColor(img_bgr,
cv2.COLOR_BGR2LAB)     mask    = build_skin_mask(img_bgr,
lms, W, H)     px_hsv  = img_hsv[mask == 255]     px_lab  =
img_lab[mask == 255]     # Oiliness = HSV saturation     avg_oil
= float(np.mean(px_hsv[:, 1]) / 255)     # Redness = LAB a*
channel (skin-tone independent)     a_vals  = px_lab[:,
1].astype(float)     avg_red = float(np.clip((np.mean(a_vals) -
128) / 40.0, 0, 1))
    # Dryness = inverted HSV value     avg_dry = float(1.0 -
np.mean(px_hsv[:, 2]) / 255)     # Pigmentation = ratio of
dark pixels (LAB L* < 100)     avg_pig =
float(np.sum(px_lab[:, 0] < 100) / len(px_lab))
    # Texture = Laplacian standard deviation
avg_tex = float(np.clip(np.std(lap_px) / 80, 0, 1))
```

The use of the LAB a* channel for redness detection is a key improvement over RGB-based approaches. The a* channel measures red-green chrominance independently of skin lightness (L* channel), making it inherently skin-tone agnostic and eliminating false positives on darker skin tones.

Skin Type Classification

The classify_skin_type() function combines YOLO model output with pixel redness to detect sensitive skin, which the model was not trained to classify:

```
def classify_skin_type(model_output, avg_red, avg_oil, avg_dry):
base = model_output.lower().strip()     is_sensitive = avg_red >
0.25     if not is_sensitive:
        return model_output.capitalize()
    # High redness detected — add Sensitive modifier     if
base == 'combination': return 'Combination-Sensitive'
elif base == 'oily':      return 'Oily-Sensitive'     elif base
== 'dry':       return 'Dry-Sensitive'     elif base == 'normal':
return 'Sensitive'
```

This hybrid approach allows the YOLO model to handle the four base skin types (Normal, Dry, Oily, Combination) while the pixel analysis layer adds the Sensitive modifier based on objective redness measurement.

Skin Health Score Computation

The compute_score() function generates a 0-100 health score from detected issues and metric magnitudes:

```
def compute_score(issues, avg_red, avg_pig, avg_oil, avg_dry):
    score = 100     score -= len([i for i in issues if i !=
'Healthy Skin']) * 6     score -= avg_red * 18     score -=
avg_pig * 12     score -= avg_oil * 8     score -= avg_dry
* 8     return max(20, min(100, int(score)))
```

4.2 BACKEND — MAKEUP DETECTION (/validate_image)

The /validate_image endpoint detects four categories of makeup using MediaPipe FaceMesh landmarks before allowing analysis to proceed:

```
# 1. LIPSTICK — lip colour vs cheek reference lip_vs_cheek_red
= (lr - cr)
lip_saturation   = (max(lr,lg,lb) - min(lr,lg,lb)) / (max(lr,lg,lb) + 1)
if is_dark_skin:     has_lipstick = lip_vs_cheek_red > 12 and
lip_saturation > 0.15 else:     has_lipstick = lip_vs_cheek_red >
10 and lip_saturation > 0.25

# 2. EYE MAKEUP — eyelid brightness vs skin brightness
lid_vs_cheek = skin_brightness - lid_brightness
eye_brightness_thresh = 40 if is_dark_skin else 60 if lid_vs_cheek >
eye_brightness_thresh and lid_sat > eye_sat_thresh:
makeup_details.append('eye_makeup')

# 3. FOUNDATION — colour variance across facial zones if
var < foundation_thresh:  # low variance = even coverage
makeup_details.append('foundation')

# SCORING: score >= 2 = makeupDetected, score >= 4 = heavyMakeup
score = 0 if 'lipstick'   in makeup_details: score += 2 if 'eye_makeup' in
makeup_details: score += 2 if 'foundation' in makeup_details: score +=
1 if 'blush'      in makeup_details: score += 1
```

4.3 FRONTEND — ProcessingPage.jsx

The ProcessingPage handles image compression, upload, timeout management, and fallback logic:

```
// Image compression before upload
async function compressImage(file, maxWidth = 800, quality = 0.75) {
const scale = Math.min(1, maxWidth / img.width);
canvas.toBlob(blob => resolve(new File([blob], file.name,         { type:
'image/jpeg' })), 'image/jpeg', quality); }

// 30-second timeout with AbortController const
controller = new AbortController();
const timeout = setTimeout(() => controller.abort(), 30000);
const res = await fetch(`${COLAB_API}/predict`, {
```

```
method: 'POST',     headers: { 'ngrok-skip-browser-
warning': '1' },     body: formData,     signal:
controller.signal,
});
```

If the Flask API is unreachable or times out, the system falls back to client-side pixel analysis using the analyzeImagePixels() function in skinAnalysis.js, which uses the browser Canvas API to compute skin metrics without any server call.

4.4 FRONTEND — skinAnalysis.js (Client-Side Fallback)

The client-side pixel analysis engine provides offline skin issue detection using the HTML5 Canvas API:

```
// Oiliness: high brightness + low channel variance = shiny
function oilScore(rg) {
   const v = Math.abs(rg.r-rg.g)+Math.abs(rg.g-rg.b);     return
rg.bright>150&&v<25 ? Math.min(1,(rg.bright-150)/80) : 0; }

// Redness: R significantly higher than G and B
function redScore(rg) {     return Math.max(0, (rg.r -
rg.g*0.9 - rg.b*0.4) / 200); }
```

The fallback also implements anime/cartoon detection by measuring pixel variance — real photos have high natural variance from texture, shadows, and noise, while anime flat fills produce very low variance values.

4.5 FRONTEND — Dynamic Backend URL

To handle ngrok URL changes without redeploying Cloudflare Pages, the colabApi.js utility reads the current backend URL from Firestore at runtime:

```
// colabApi.js — reads ngrok URL from Firestore config/colab
let cachedUrl = null; let cacheTime = 0;
const CACHE_TTL = 5 * 60 * 1000; // 5 minutes

export async function getColabUrl() {
   if (cachedUrl && Date.now() - cacheTime < CACHE_TTL)
return cachedUrl;     const doc = await getDoc(doc(db,
'config', 'colab'));     cachedUrl = doc.data().apiUrl;
cacheTime = Date.now();     return cachedUrl;
}
```

4.6 FRONTEND — Phone QR Scan

The QR scan feature uses a session-based polling architecture to allow phone camera capture:

```
// PhoneScanQR.jsx — poll Flask every 2 seconds pollRef.current =
setInterval(async () => {     const res = await
fetch(`${COLAB_API_BASE}/phone_poll/${sessionId}`,
```

```
        { headers: { 'ngrok-skip-browser-warning': '1' } });
const data = await res.json();      if (data.status ===
'done' && data.result) {
clearInterval(pollRef.current);
onResult(data.result, data.imageDataUrl);
    }
}, 2000);
```

5. Software Testing

Software testing is performed to ensure that all modules of Advanced Beauty Care function correctly and meet user requirements. Testing verifies accurate skin analysis computations, proper image validation, correct product matching, and smooth interaction between all frontend and backend components.

5.1 UNIT TESTING

Unit testing is performed to verify the functionality of individual modules independently. Each module is tested with different inputs to ensure correct outputs. The following test cases validate the correctness and reliability of individual modules.

Test ID	Module	Test Description	Input	Expected Result	Status
UT-1	Authentication	Verify user sign-in with Google	Valid Google account	User logged in, redirected to home	Pass
UT-2	Authentication	Verify email/password login	Valid email & password	Login successful	Pass
UT-3	Authentication	Invalid login credentials	Wrong password	Error message displayed	Pass
UT-4	Analysis	Upload valid bare-skin photo	Clear selfie, no makeup	Validation passes, proceeds to processing	Pass
UT-5	Validation	Reject anime/cartoon image	Anime illustration image	Blocked: 'No real human face found'	Pass
UT-6	Validation	Detect makeup in photo	Photo with lipstick & eye makeup	Warning: 'Makeup detected'	Pass
UT-7	Validation	Reject dark/underexposed image	Very dark photo	Blocked: 'Image too dark'	Pass
UT-8	Processing	Image compression before upload	4MB phone photo	Compressed to ~400KB (75% quality, 800px max)	Pass
UT-9	Processing	Backend timeout fallback	Flask server offline	Falls back to pixel analysis, shows offline notice	Pass
UT-10	Results	Skin type classification	Fair-skin photo with oiliness	Correct skin type returned by YOLO	Pass
UT-11	Results	Acne type classification	Photo with visible papules	Acne Type = Papules, Severity = Mild	Pass
UT-12	Results	Skin health score range	Any valid photo	Score between 0-100, min 20	Pass
UT-13	Products	Product filter by skin type	Filter: Sensitive	Only sensitive-suitable products shown	Pass
UT-14	Products	Add to cart	Click Add on product card	Product added, cart count updates	Pass
UT-15	Ingredient Scanner	Search ingredient by name	Search: 'Niacinamide'	Ingredient details and products displayed	Pass

5.2 INTEGRATION TESTING

Integration testing verifies the interaction between different modules of the system. It ensures that data flows correctly between components such as the image upload, validation, Flask backend, results rendering, and Firebase storage.

Test ID	Modules	Test Description	Expected Result	Status
IT-1	Analysis → Validation → Processing	Full pipeline: upload → validate → process → result	Complete analysis result displayed in ResultPage	Pass
IT-2	Flask → YOLOv8 → Pixel Analysis	Backend processes image through all 3 YOLO models + pixel analysis	JSON response with skinType, acneType, severity, issues, metrics	Pass
IT-3	Processing → ResultPage	Analysis result passed from ProcessingPage to ResultPage	All fields (score, issues, regStatus, metrics) render correctly	Pass
IT-4	ResultPage → Firestore	Scan saved to Firestore after analysis	Entry appears in Dashboard scan history	Pass
IT-5	ResultPage → ProductsPage	Navigate from result to matched products	Products filtered for detected skin type and issues	Pass
IT-6	QR Scan → Flask → ResultPage	Phone uploads photo via QR → Flask processes → laptop shows result	Full result shown on laptop after phone scan	Pass
IT-7	ManualInputPage → Recommendations	7-step questionnaire generates offline plan	Matched products, AM/PM routine, remedies generated without API	Pass
IT-8	Dashboard → RecheckPage	Scan history loaded from Firestore on dashboard	All past scans displayed with score delta	Pass

6. Sample Inputs and Outputs

This chapter presents the input and output screens of Advanced Beauty Care, demonstrating the working of different modules in the system. The screenshots illustrate how users interact with the application and how the system processes and displays skin analysis data.

Screen 1: Home Page

- Provides the main entry point to the application.
- Offers navigation to Skin Analysis, Products, Ingredients, Recheck, About, and Dashboard.
- Displays two primary entry modes: Start Skin Analysis and Manual Input.
- Shows a product showcase on the hero section.

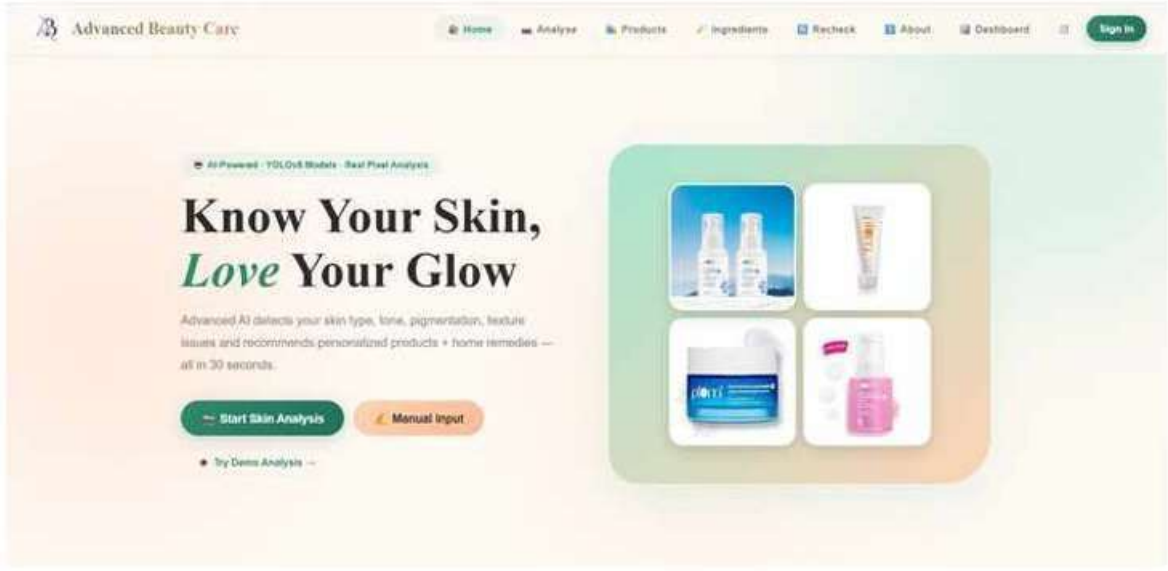

Figure 6.1 – Home Page

Screen 2: Login / Sign In

- Allows users to sign in with email/password or Google OAuth.
- Firebase Authentication handles session management.
- New users can register via the Sign Up link.

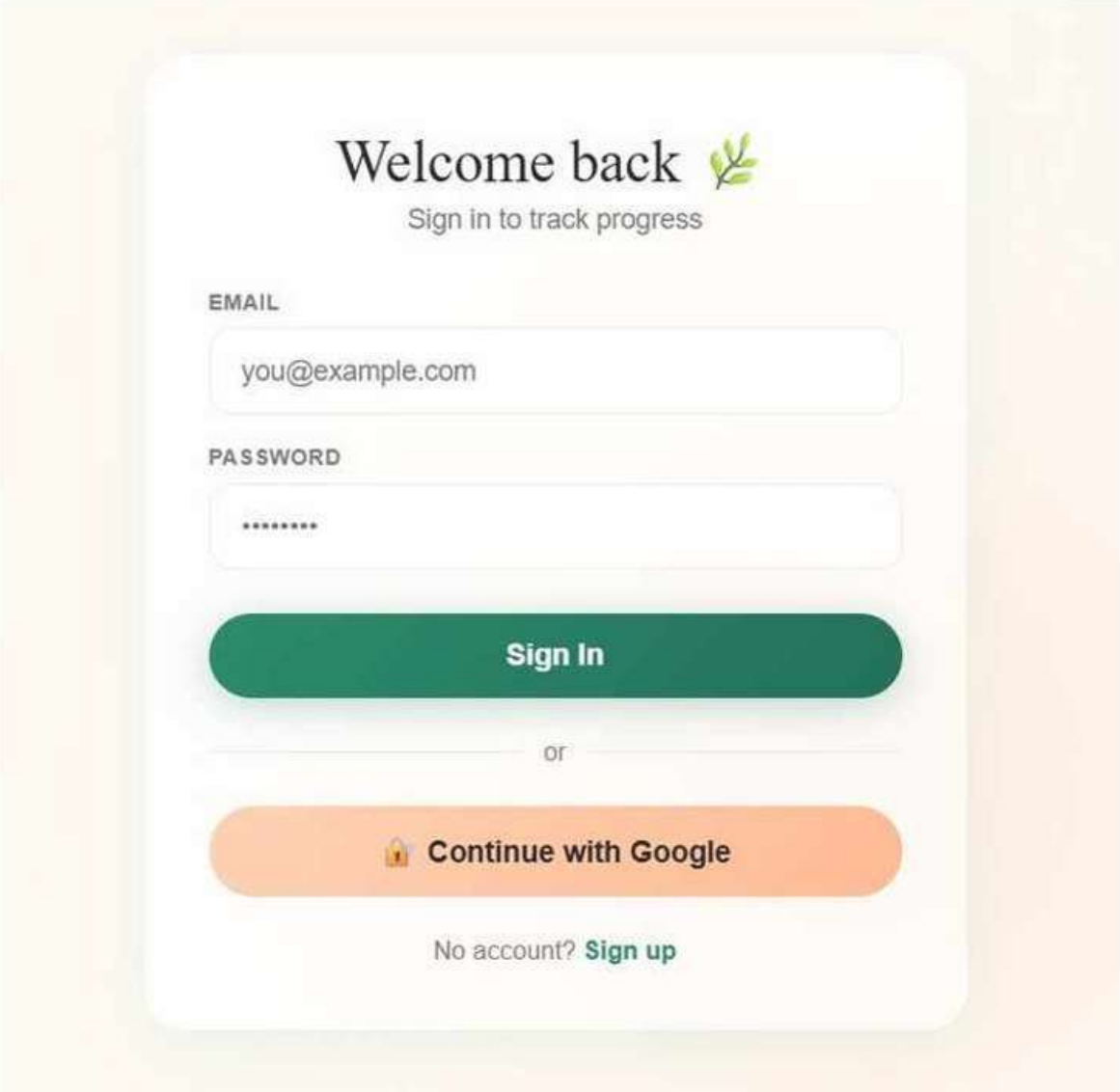

Figure 6.2 – Login Page

Screen 3: Analysis Page — Photo Upload

- Allows users to upload a clear bare-skin selfie via drag-and-drop or file browser.
- Provides photography tips: remove makeup, good lighting, neutral face, no filters.
- Offers an alternative phone camera option via QR code scan.
- Links to full Photography Guidelines popup.

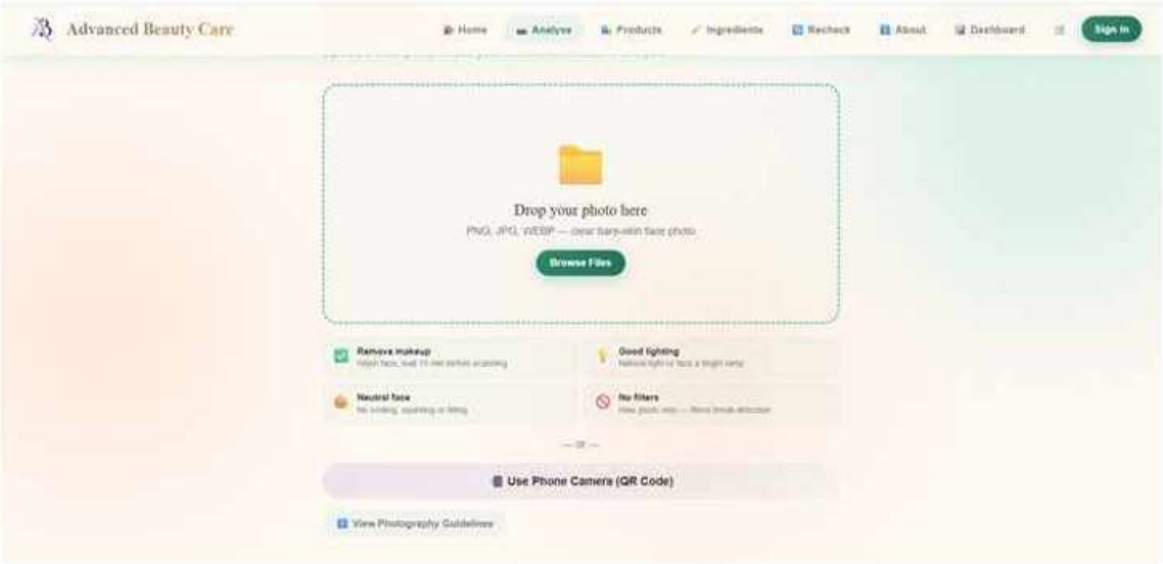

Figure 6.3 – Analysis / Photo Upload Page

Screen 4: Image Validation — Anime/Cartoon Rejection

- Demonstrates the image validation system in action.
- An anime illustration is correctly rejected: 'No real human face found.'
- Other checks (no makeup, good lighting, portrait dimensions) pass.
- User is prompted to upload a real bare-skin selfie.

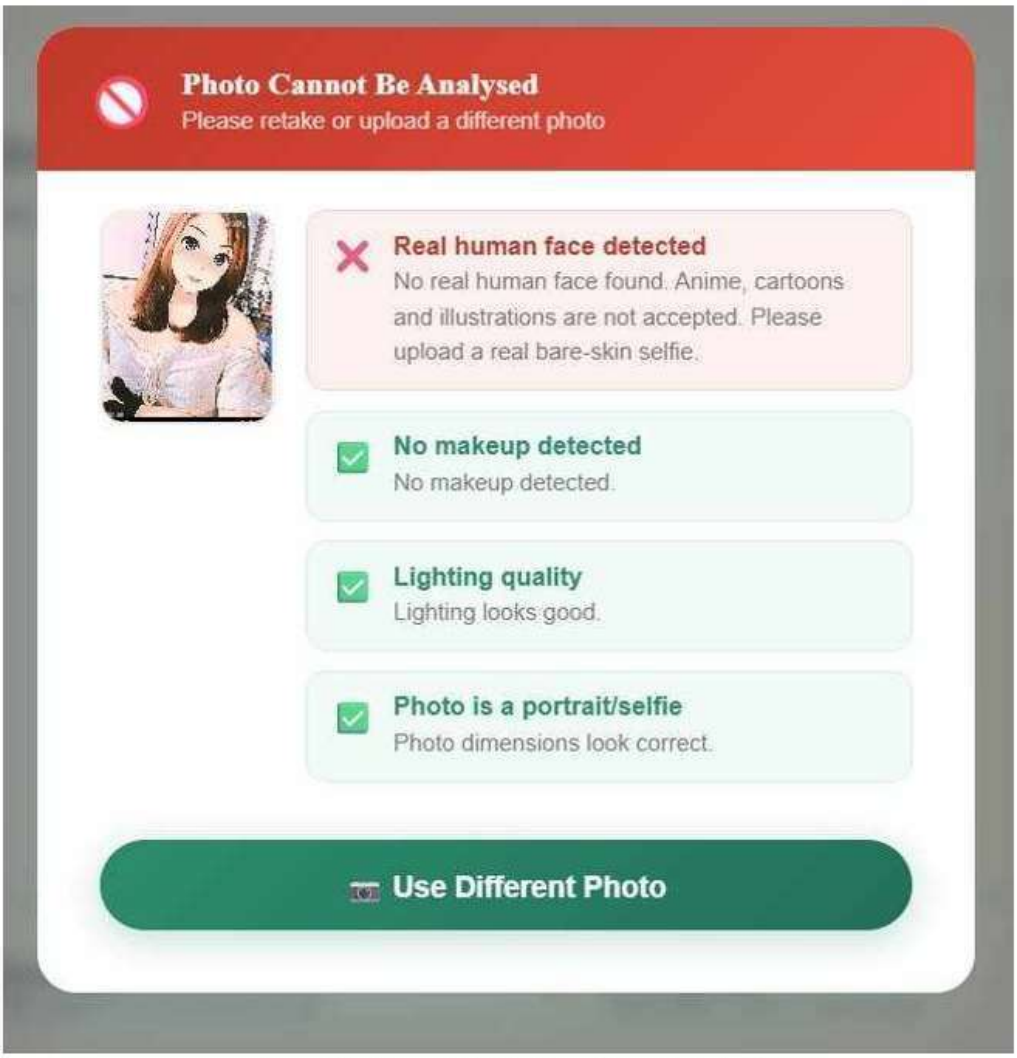

Figure 6.4 – Image Validation Error (Anime Rejection)

Screen 5: Processing Page

- Shows an animated progress bar and step-by-step AI analysis status.
- Steps include: image quality check, face detection, makeup check, facial region segmentation, pixel analysis, redness detection, and model inference.
- Displays current upload status (e.g. 'Uploading to server...').
- Handles 30-second timeout with graceful fallback to local pixel analysis.

Figure 6.5 – Processing Page (AI Analysis in Progress)

Screen 6: Result Page — Skin Analysis Report

- Displays the full skin analysis report with health score ring (67/100).
- Shows detected skin type (Sensitive), confidence (91.2%), skin tone (Medium, Neutral Undertone).
- Lists detected issues: Oiliness, Redness, Acne-Prone, Sensitivity.
- Provides JSON Report and PDF Report download options.

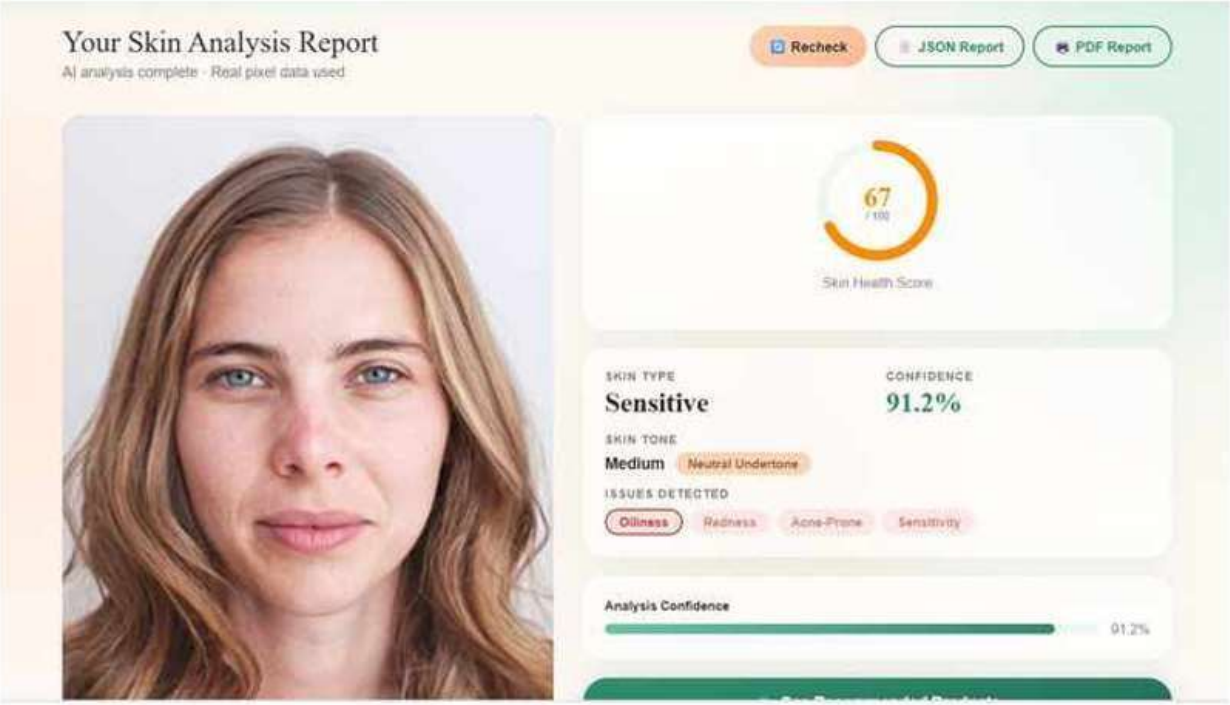

Figure 6.6 – Result Page (Skin Health Score and Analysis)

Screen 7: Result Page — YOLOv8 Detection + Product Picks

- Shows YOLOv8 model outputs: Acne Type = Papules, Severity = Mild, Skin Type = Sensitive.
- Displays facial region analysis: Forehead (Slight Redness), Left/Right Cheek (Redness), Nose (Normal), Chin (Normal).
- Explains why products were selected based on detected issues.
- Shows 'Your Personalised Picks' — 27 products matched for Sensitive skin with Oiliness, Redness, Acne-Prone.

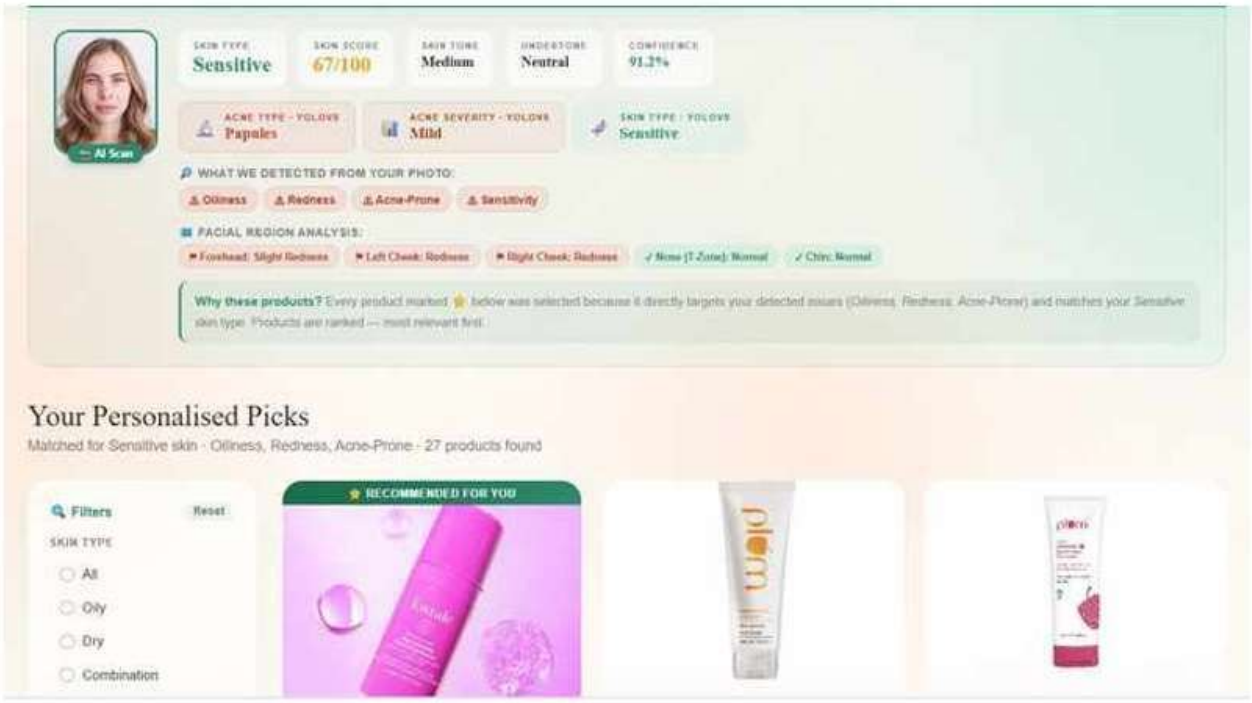

Figure 6.7 – Result Page (YOLOv8 Outputs + Personalised Product Picks)

Screen 8: Result Page — AM/PM Routine

- Displays personalised morning and night skincare routines.
- Morning: Cleanser (Salicylic Acid) → Toner (Niacinamide) → Serum (Niacinamide 10%) → Moisturiser → SPF 50+.
- Night: Double Cleanse → Exfoliant (BHA 2%) → Treatment Serum → Eye Cream → Moisturiser.
- Each step includes the recommended product type and rationale.

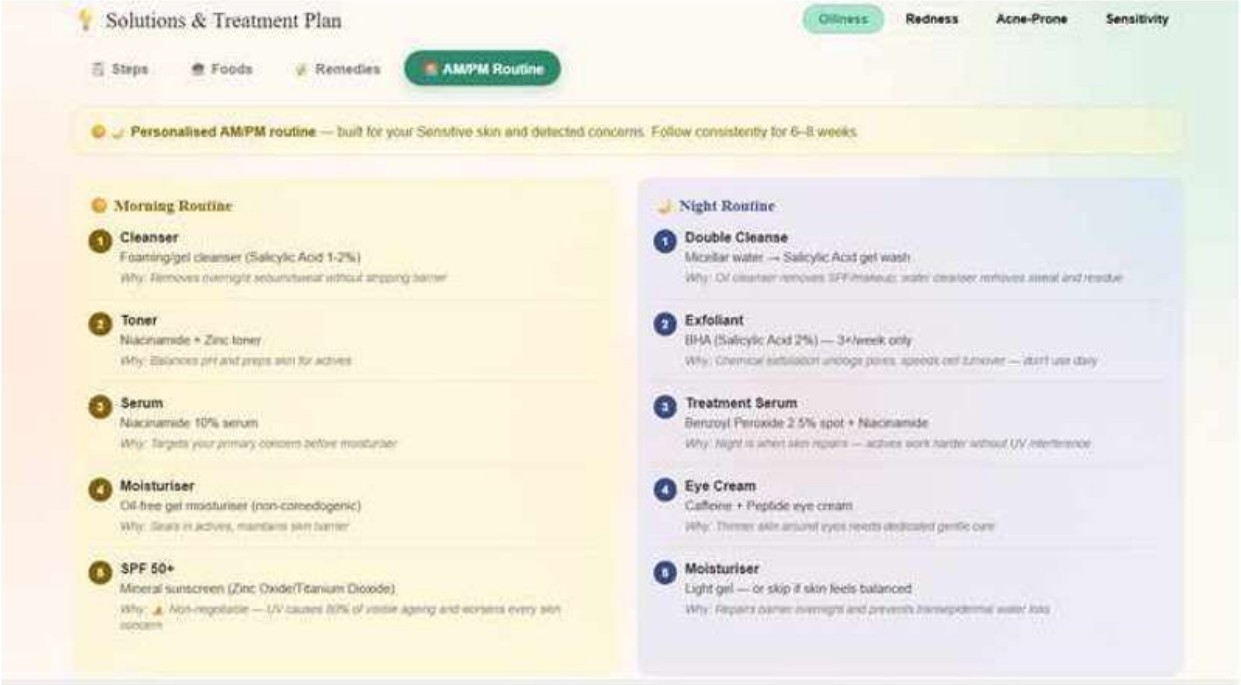

Figure 6.8 – Result Page (Personalised AM/PM Routine Tab)

Screen 9: Manual Input — Step 1 (Skin Goals)

- First step of the 7-step questionnaire wizard.
- Users select their skin goals: Glass Skin, Natural Glow, Acne-Free, Anti-Aging, Even Tone, Deep Hydration, Calm Sensitive, Minimize Pores.
- Progress bar at top shows Step 1 of 7.
- Generates a personalised plan without requiring a photo upload.

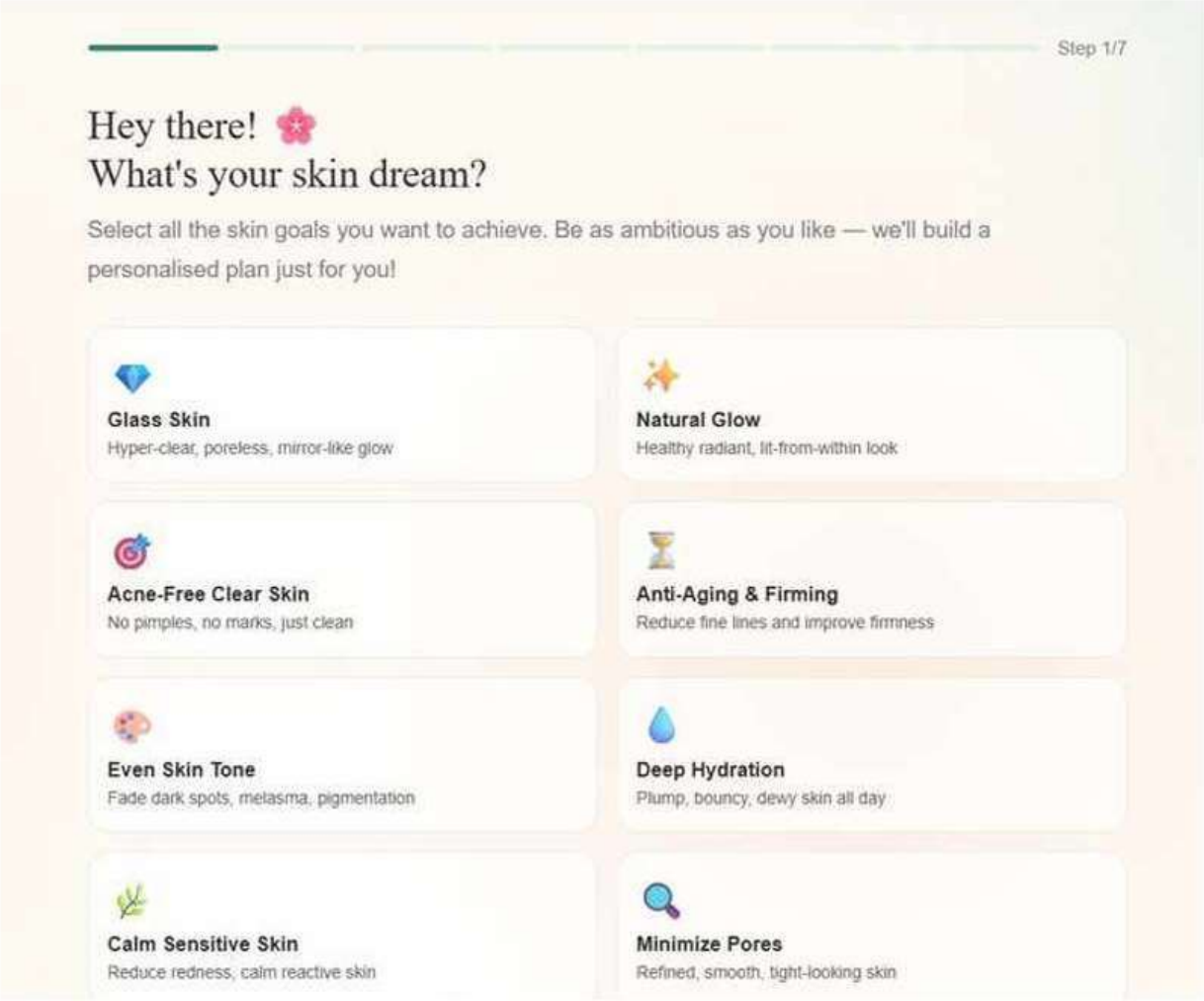

Figure 6.9 – Manual Input Page (Step 1 – Skin Goals)

Screen 10: Products Page

- Displays all 124 curated Indian skincare products.
- Filter panel allows filtering by skin type (All/Oily/Dry/Combination/Normal/Sensitive) and brand.
- Each product card shows brand, name, category, skin type tags, price, and Add/Buy/Wishlist buttons.
- Products include brands: Plum, Minimalist, WishCare, Dot & Key, Mamaearth, Himalaya, Foxtale, CeraVe.

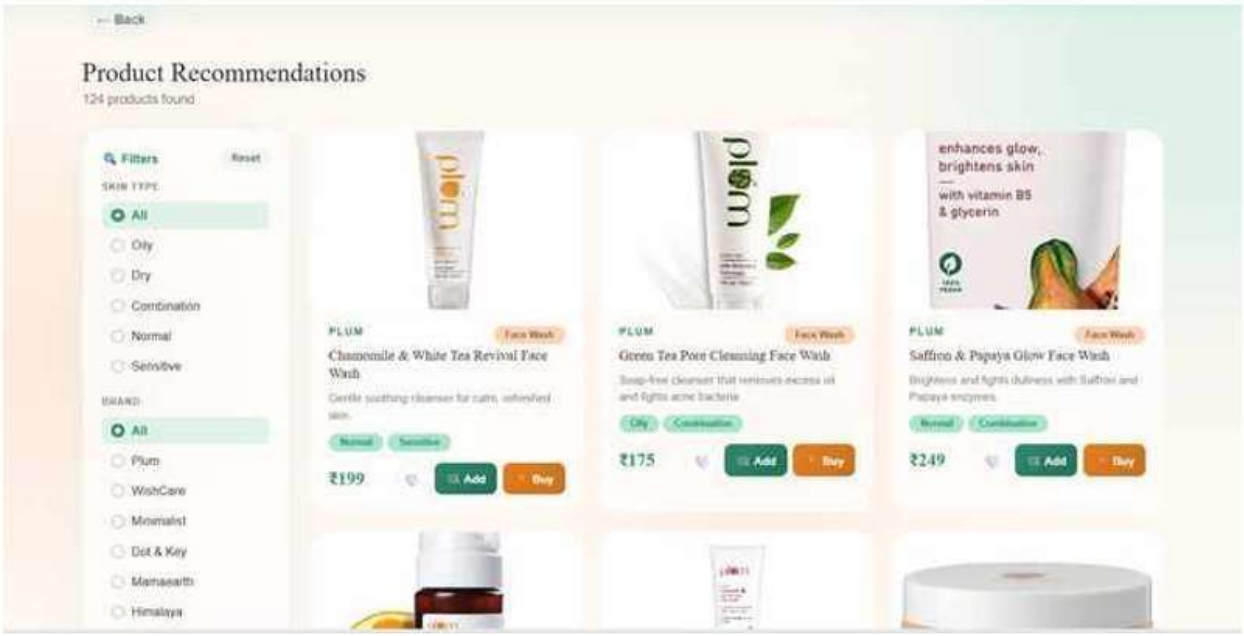

Figure 6.10 – Product Recommendations Page (124 Products)

Screen 11: Product Detail Page

- Shows full product details for Plum Chamomile & White Tea Revival Face Wash (₹199).
- Displays key ingredient cards: Chamomile Extract, White Tea Extract, Glycerin.
- Each ingredient card explains 'What it does' and 'How it works' at molecular level.
- Shows skin suitability tags, star rating (4.7), and customer reviews.

Figure 6.11 – Product Detail Page (Ingredients & Usage)

Screen 12: Ingredient Scanner

- Searchable database of skincare ingredients.
- Shown: AHA + BHA Exfoliant Blend (10%) — tagged for Acne-Prone, Dark Spots, Uneven Texture, Oily Skin.
- Explains 'What It Does' and 'How It Works' at molecular level.
- Shows products containing this ingredient (Minimalist, Foxtale).

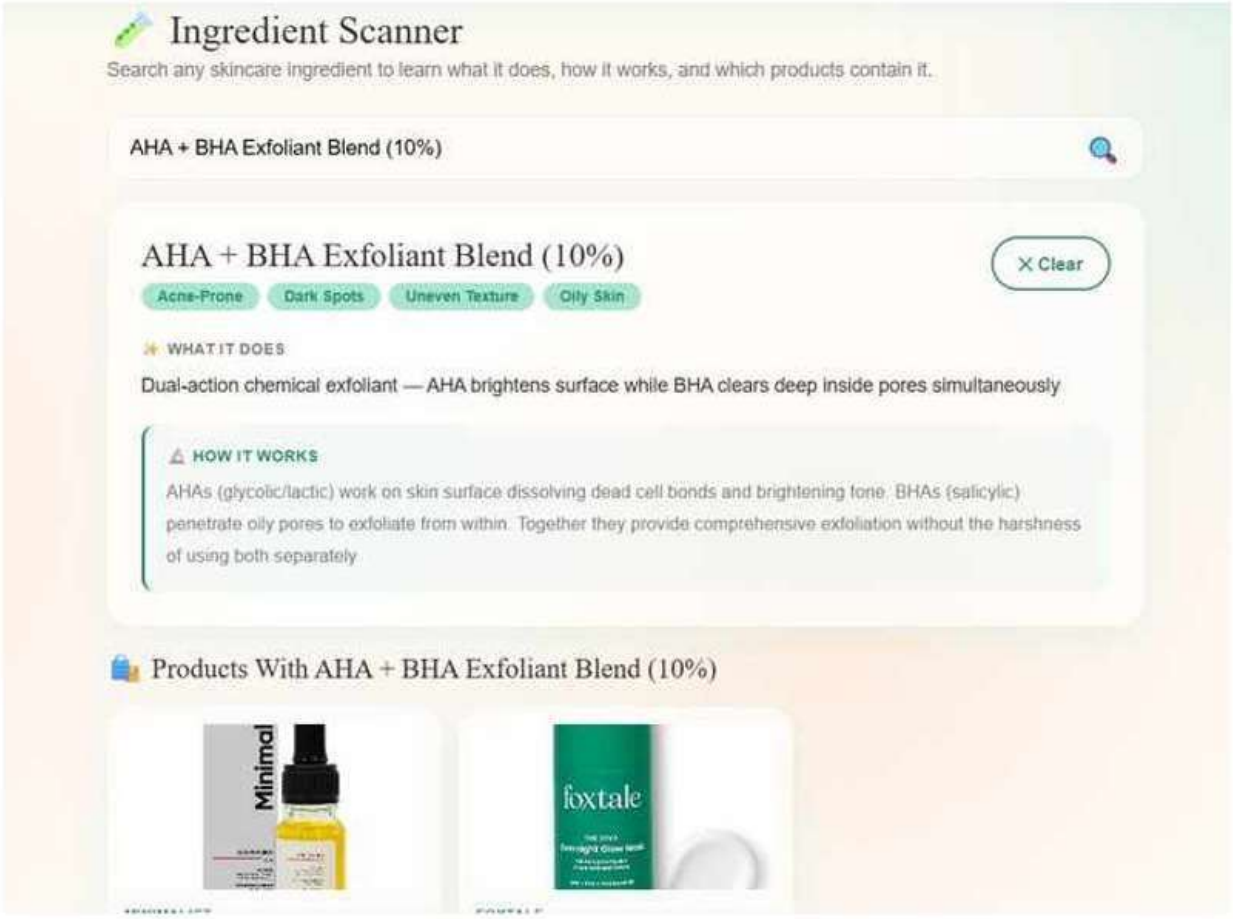

Figure 6.12 – Ingredient Scanner Page

Screen 13: About Page — How It Works

- Explains the 5-step AI analysis pipeline visually.
- Step 1: Upload Photo → Step 2: AI Validation → Step 3: YOLOv8 Analysis → Step 4: Pixel Analysis → Step 5: Results.
- Each step describes the technology used (MediaPipe + Haar Cascade, YOLOv8, OpenCV).

Figure 6.13 – About Page (How It Works)

Screen 14: Dashboard — Analysis History

- Shows user's scan history with a Skin Score Progress line chart.
- Displays 5 scans showing score trend over time (score ~69, no significant change).
- Sidebar provides navigation to History, Cart, Wishlist, Recheck, and Settings.
- Each scan entry shows skin type, detected issues, score, and score delta.

Figure 6.14 – Dashboard Page (Scan History & Score Progress)

Screen 15: Skin Recheck & Progress

- Displays scan history timeline with timestamps.
- Shows 4 recent scans (3/4/2026 at various times) with score change indicator.
- Take New Scan button navigates to Analysis page for a fresh scan.
- Used alongside the routine checkup form to analyse current products.

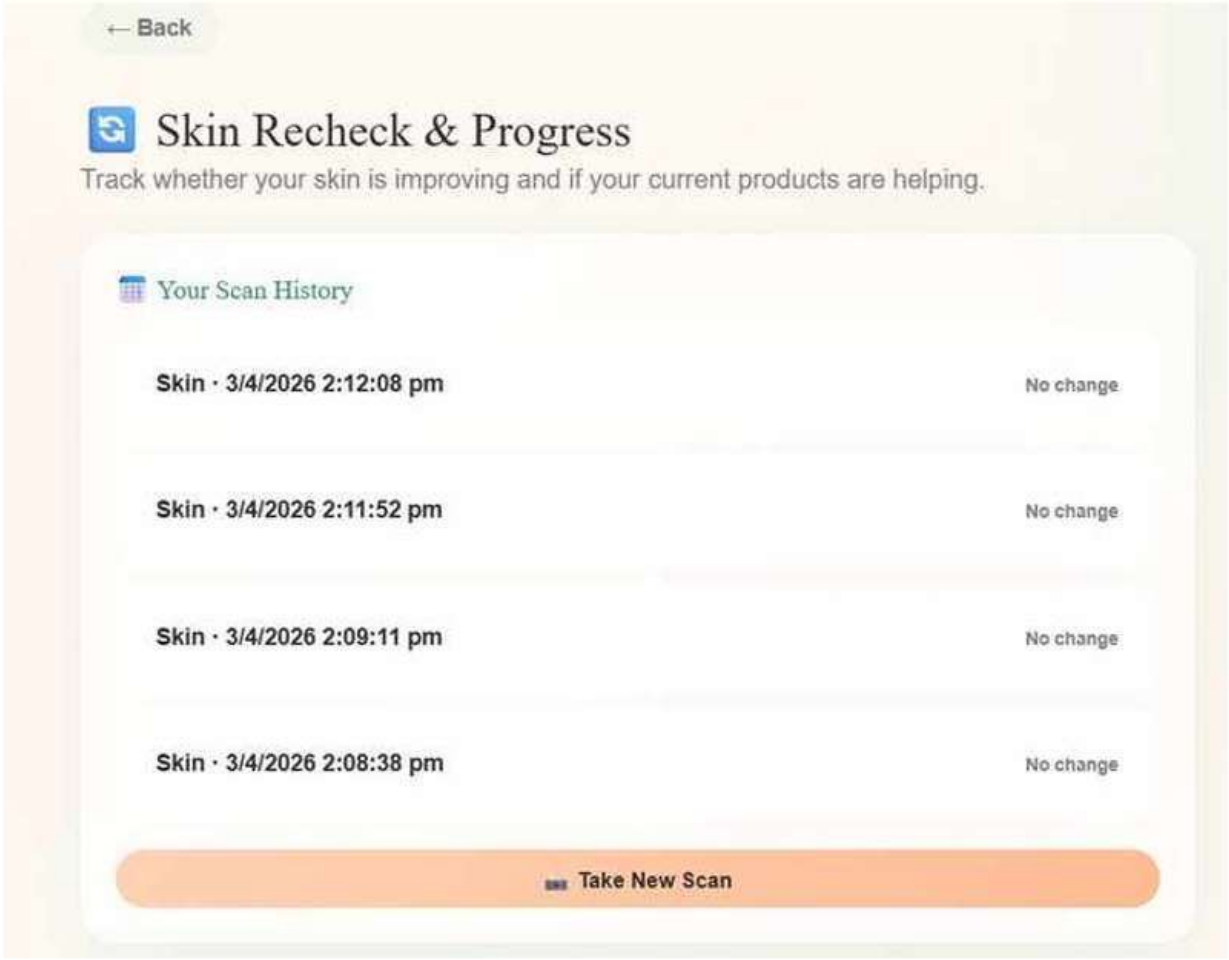

Figure 6.15 – Skin Recheck & Progress Page

7. Conclusion

The development of Advanced Beauty Care – AI-Powered Skin Analysis Platform successfully demonstrates an efficient and intelligent approach to personalised skincare. The system integrates three independently trained YOLOv8 classification models, a Python Flask REST API with OpenCV pixel analysis and MediaPipe landmark detection, and a React frontend — delivering a comprehensive skin analysis from a single photograph in under 30 seconds.

The system includes several important technical contributions:

- LAB a* channel for redness detection — skin-tone independent, eliminates bias against darker skin tones.
- Landmark-based skin mask with bindi/lip/eye exclusions — ensures pixel metrics sample only skin pixels.
- Hybrid sensitive skin detection — YOLO base type + pixel redness modifier for compound types.
- Multi-category makeup detection using MediaPipe FaceMesh — lipstick, eye makeup, foundation, blush with dark-skin adjusted thresholds.
- Anime/cartoon rejection using pixel variance analysis — rejects non-real-photo inputs reliably.
- Dynamic backend URL via Firestore config — eliminates redeployment requirement on Colab restart.
- 30-second timeout with graceful pixel fallback — ensures the app works even when the backend is offline.

The inclusion of a 124-product Indian skincare database, a 7-step manual input questionnaire, an ingredient scanner, scan history tracking with score progress visualisation, and exportable PDF/JSON reports makes Advanced Beauty Care a comprehensive skincare companion rather than a simple classifier.

The project is implemented using a modern, scalable architecture with React 18, Flask, YOLOv8, Firebase, and Cloudflare Pages, ensuring smooth integration between frontend, backend, and cloud services. Through comprehensive unit and integration testing, all modules have been verified to function correctly and reliably.

Overall, Advanced Beauty Care transforms basic skincare curiosity into an intelligent, interactive, and datadriven experience. It helps users understand their skin objectively, make informed product choices, and build consistent skincare routines with measurable progress over time.

8. Scope and Future Enhancements

8.1 Current Scope

The current version of Advanced Beauty Care supports face photograph-based skin analysis for any skin tone, classification of skin type (Normal, Dry, Oily, Combination, and Sensitive variants), acne type (Papules, Pustules, Nodules, Blackheads, Whiteheads), and acne severity (Mild, Moderate, Severe). It detects and quantifies six skin concerns: oiliness, redness, dryness, pigmentation, texture roughness, and acne-proneness, and derives additional issues (dark spots, dehydration, sensitivity) from metric combinations. Personalised product recommendations are generated with full ingredient explanations. Scan history is tracked per user via Firestore, enabling longitudinal comparison across sessions. A manual skin consultation wizard provides an

alternative pathway for users who prefer not to upload a photograph. A phone-to-desktop scan flow via QR session ID enables mobile camera use without a dedicated mobile app.

Advanced Beauty Care provides a strong foundation for AI-powered personalised skincare. However, there is significant scope for further enhancement to improve system intelligence, accuracy, scalability, and user experience.

8.2 Technical Enhancements

- Persistent Backend Deployment – Migrate Flask backend from Google Colab to a persistent cloud service (Google Cloud Run, AWS Lambda, or Render) to eliminate the manual session startup requirement and provide 24/7 availability.
- Improved Skin Type Model – Retrain the skin type model on a larger, more diverse dataset with additional Sensitive class examples to improve accuracy beyond the current 81.5%.
- On-Device Inference – Explore TensorFlow.js or ONNX.js to run lightweight classification models directly in the browser, eliminating network dependency for basic skin type detection.
- Wrinkle & Fine Line Detection – Add edge detection and depth estimation to detect fine lines and wrinkles as additional skin issues.
- 3D Facial Mapping – Use MediaPipe's 3D landmark coordinates to build a 3D skin condition map with depth-aware analysis.

8.3 Feature Enhancements

- Dynamic Product Database – Integrate live product pricing and availability via affiliate APIs (Nykaa, Amazon, Flipkart) to replace the static curated catalog.
- Weekly Scan Reminders – Implement push notifications (via Firebase Cloud Messaging) to remind users to scan weekly and track progress.
- Ingredient Conflict Checker – Build a tool that analyses a user's current product list and flags ingredient combinations that should not be used together (e.g. Retinol + Vitamin C, AHA + Retinol).
- Before/After Comparison – Enable side-by-side comparison of scan images with annotated issue markers across two time points.
- Multilingual Support – Add Hindi, Tamil, Telugu, and Kannada language support for broader Indian market reach.

8.4 AI & Analysis Improvements

- Dermatologist Validation – Partner with dermatologists to validate model outputs and build a feedback loop for continuous model improvement.
- Acne Severity Tracking – Track acne severity scores over multiple scans to measure treatment effectiveness objectively.
- Personalised Ingredient Recommendations – Use user scan history to learn which ingredients have historically improved their score and weight those in product recommendations.

Mobile Application – Develop a React Native or Flutter mobile app for iOS and Android to provide a native camera experience and offline analysis capability.

9. Appendix

Appendix A: AI Model Performance Metrics

The following table presents the complete evaluation metrics for all three YOLOv8 models as reported on the held-out test set from the Kaggle training datasets. All models use YOLOv8 in classification mode (YOLOv8cls).

Model	Accuracy	Precision	Recall	F1 Score	Cohen's κ	AUC
Skin Type · YOLOv8	0.815	0.815	0.815	0.814	0.778	0.950
Acne Type · YOLOv8	0.748	0.750	0.748	0.746	0.672	0.913
Acne Severity · YOLOv8	0.991	0.992	0.991	0.991	0.950	0.999

Appendix B: Pixel Metric Thresholds (Backend)

The following thresholds are applied in flask_predict_final.py to classify detected skin issues from computed pixel metrics:

Metric	Issue Triggered	Moderate Threshold	High/Severe Threshold
avgOil (HSV S/255)	Oiliness	> 0.22	> 0.40 = High
avgRed (LAB a* normalised)	Redness	> 0.25	> 0.50 = Severe
avgPig (LAB L* < 100 ratio)	Pigmentation	> 0.20	> 0.40 = Significant
avgDry (1 − HSV V/255)	Dryness	> 0.30	> 0.50 = Severe
avgTex (Laplacian std/80)	Uneven Texture	> 0.30	> 0.50 = Rough
avgRed > 0.20 AND avgOil > 0.22	Acne-Prone	Both conditions met	Uses YOLO severity
avgPig > 0.15	Dark Spots	> 0.15	> 0.35 = Significant
avgOil < 0.12 AND avgDry > 0.30	Dehydration	Both conditions met	Moderate only
avgRed > 0.25	Sensitive modifier	Added to YOLO skin type	Threshold for Sensitive tag

Appendix C: Flask API Endpoints

Method	Endpoint	Description
POST	/predict	Main analysis pipeline — accepts image, returns full skin analysis JSON
POST	/validate_image	Face detection + makeup detection — called before /predict
POST	/phone_upload	Receives photo from phone QR scan, runs analysis in background thread
GET	/phone_poll/:id	Long-polling — returns session status and result when ready
GET	/health	Health check — returns model count and MediaPipe version

Appendix D: Product Database Summary

The product catalog contains 124 Indian skincare products across 6 categories. Each product entry contains: name, brand, category, price (INR), description, usage instructions, compatible skin types, target issues, and key ingredients with benefit, mechanism (howWorks), and skin suitability tags.

Category	Count	Brands Included
Face Wash	~25	Plum, Mamaearth, Himalaya, WishCare, Foxtale
Serum	~30	Minimalist, Dot & Key, WishCare, Foxtale, Plum
Moisturizer	~25	CeraVe, Plum, Minimalist, Dot & Key, Mamaearth
Sunscreen	~20	Minimalist, Dot & Key, Plum, WishCare
Toner	~12	Minimalist, Plum, Foxtale, Mamaearth
Mask	~12	Plum, Dot & Key, Himalaya, WishCare

Appendix E: Abbreviations and Acronyms

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the ROC Curve
BHA	Beta Hydroxy Acid
CLAHE	Contrast Limited Adaptive Histogram Equalisation
EMU	English Metric Units (used in DOCX for image sizing)
HSV	Hue, Saturation, Value (color space)

LAB	CIELAB color space (L* = lightness, a* = red-green, b* = blue-yellow)
ML	Machine Learning
SPF	Sun Protection Factor
UI	User Interface
UX	User Experience
YOLOv8	You Only Look Once version 8 (object detection/classification model)

Appendix F: System Requirements

Component	Requirement
Browser (Frontend)	Chrome 100+, Firefox 100+, Safari 15+, Edge 100+
Internet Connection	Required for Flask API; offline fallback available
Google Colab (Backend)	GPU runtime (T4 or better) for YOLOv8 inference
Python Version	Python 3.9+ for Flask backend
Node.js (Development)	Node 18+ for React/Vite development
Firebase Project	Firebase Authentication + Cloud Firestore enabled

Appendix G: List of Figures

Figure No.	Title / Description	Chapter
Figure 6.1	Home Page – Landing and Navigation	Chapter 6
Figure 6.2	Login Page – Sign In / Google Auth	Chapter 6
Figure 6.3	Analysis Page – Photo Upload Interface	Chapter 6
Figure 6.4	Image Validation Error – Anime/Cartoon Rejection	Chapter 6
Figure 6.5	Processing Page – AI Analysis Progress	Chapter 6
Figure 6.6	Result Page – Skin Health Score and Analysis	Chapter 6
Figure 6.7	Result Page – YOLOv8 Outputs and Product Picks	Chapter 6
Figure 6.8	Result Page – Personalised AM/PM Routine Tab	Chapter 6
Figure 6.9	Manual Input Page – Step 1 Skin Goals	Chapter 6
Figure 6.10	Products Page – 124 Product Catalog	Chapter 6
Figure 6.11	Product Detail Page – Ingredients and Usage	Chapter 6
Figure 6.12	Ingredient Scanner Page	Chapter 6

Figure 6.13	About Page – How It Works Pipeline	Chapter 6
Figure 6.14	Dashboard Page – Scan History and Score Progress	Chapter 6
Figure 6.15	Skin Recheck and Progress Page	Chapter 6

10. Bibliography and References

[1] Jocher, G. et al. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[2] Lugaresi, C. et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.
[3] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
[4] Banks, A. & Porcello, E. (2020). Learning React. O'Reilly Media.
[5] Russell, S. & Norvig, P. (2010). Artificial Intelligence: A Modern Approach. Prentice Hall.
[6] Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.
[7] Sommerville, I. (2015). Software Engineering. Pearson Education.
[8] Grinberg, M. (2018). Flask Web Development. O'Reilly Media.
[9] Firebase Team. (2024). Firebase Documentation. https://firebase.google.com/docs
[10] React Developers. (2024). React Official Documentation. https://react.dev/
[11] Ultralytics. (2024). YOLOv8 Documentation. https://docs.ultralytics.com/
[12] MediaPipe Authors. (2024). MediaPipe Solutions. https://developers.google.com/mediapipe
[13] Cloudflare. (2024). Cloudflare Pages Documentation. https://developers.cloudflare.com/pages/
[14] OpenAI. (2026). ChatGPT (Large Language Model). https://chat.openai.com/

Documentation and Web References

4. React Documentation. https://react.dev

5. Flask Documentation. https://flask.palletsprojects.com

6. Firebase Documentation. https://firebase.google.com/docs

7. Cloudflare Pages Documentation. https://developers.cloudflare.com/pages

8. ngrok Documentation. https://ngrok.com/docs

9. OpenCV LAB Color Space. https://docs.opencv.org/4.x/de/d25/imgproc_color_conversions.html 10. Ultralytics YOLOv8 Documentation. https://docs.ultralytics.com

Tools Used

11. Roboflow — Dataset preparation and annotation for YOLOv8 model training. https://roboflow.com

12. Google Colab — Cloud GPU environment for model training and Flask backend hosting. https://colab.research.google.com

13. Vite — Frontend build tool. https://vitejs.dev

14. Python 3.12, NumPy, Pillow — Core backend libraries.